

Minimum number of paths needed to cover a domain (For both flat (with ∂) and closed (without ∂) manifold)

Hridoy Ranjan Kalita

3/05/2026

Contents

1	The Problem	2
1.1	Minimum Number of Paths	2
2	Three Lower Bounds on $\text{mpc}(G)$	2
2.1	Bound 1 — Connected Components	2
2.2	Bound 2 — Bipartite Imbalance	2
2.3	Bound 3 — Leaves	3
2.4	The Combined Bound	3
3	Problem Setup — The Chessboard Grid	3
4	Partition into Balanced Subdomains	4
4.1	Construct a Spanning Tree	4
4.2	Subtree Sums	4
4.3	Cutting Subtrees to Obtain $ \delta_C $ Subdomains	4
5	Finding Hamiltonian Paths	5
5.1	Stage 1 — Greedy Warnsdorf Heuristic	5
5.2	Stage 2 — Depth-First Search with Backtracking	5
5.3	Stage 3 — Partial Path Fallback	5
5.4	Pruning for Speed	5
6	Complexity and Practical Performance	5
7	Results and discussions	6
7.1	Planar case	6
7.2	”Closed” surface case	7
8	Future Work and Some discussions	7
9	Summary	8

1 The Problem

1.1 Minimum Number of Paths

Given a graph $G = (V, E)$, a path cover is a set of vertex-disjoint paths $P_1, P_2, \dots, P_k$ such that

$$V = P_1 \cup P_2 \cup \dots \cup P_k, \quad P_i \cap P_j = \emptyset \quad \forall i \neq j.$$

The minimum path cover number is

$$\text{mpc}(G) = \min\{k : \text{such a partition exists}\}.$$

The physical interpretation: if you think of drawing G on paper without lifting the pen, then every time you do have to lift the pen you start a new path. So:

$$\text{pen lifts} = \text{mpc}(G) - 1.$$

Minimising pen lifts is the same as minimising the number of paths.

Remark 1.1. *Deciding whether $\text{mpc}(G) = 1$ (i.e. whether G has a Hamiltonian path) is NP-hard. It follows that computing $\text{mpc}(G)$ in general is NP-hard too. So instead, we focus on finding good lower bounds and building practical heuristics.*

2 Three Lower Bounds on $\text{mpc}(G)$

Even though computing $\text{mpc}(G)$ exactly is hard, we can prove several structural lower bounds. Together they give a strong floor on how many paths we need.

2.1 Bound 1 — Connected Components

If a graph has c connected components $G_1, G_2, \dots, G_c$, then any path stays inside one component. So you need at least one path per component.

Lemma 2.1. $\text{mpc}(G) \geq c$, where c is the number of connected components of G .

2.2 Bound 2 — Bipartite Imbalance

Colour every vertex either Black (B) or White (W) like a chessboard. For a connected component, let $|B|$ and $|W|$ be the counts of each colour. The imbalance of the component is $d = ||B| - |W||$.

Theorem 2.2. *Any path cover of a connected component with imbalance $d \geq 0$ requires at least d paths.*

Proof. Every path in the graph is one of two types:

- BW/WB path — it alternates colours, so $|B_i| = |W_i|$, contributing imbalance 0.
- BB/WW path — it starts and ends on the same colour, so $||B_i| - |W_i|| = 1$ contributing imbalance ± 1 .

Suppose a component is covered by k paths, of which b are BW/WB and a are BB/WW (so $a + b = k$). Then:

$$d = |B| - |W| = \sum_{i=1}^k (|B_i| - |W_i|).$$

Each term is 0 or ± 1 , and the imbalanced paths each contribute exactly ± 1 . So $d \leq k$ (since the a imbalanced paths contribute at most $a \leq k$ to the total). Hence $k \geq d$. $\square$

Summing over all components, if component j has imbalance d_j , then:

$$\text{mpc}(G) \geq \sum_{j=1}^c d_j.$$

2.3 Bound 3 — Leaves

A leaf is a vertex of degree 1. Let $L = \{v \in V : \deg(v) = 1\}$.

Theorem 2.3. $\text{mpc}(G) \geq \left\lceil \frac{|L|}{2} \right\rceil$.

Proof. Every leaf must be an endpoint of any path that covers it, because it has only one neighbour. A path has exactly two endpoints. So k paths have at most $2k$ endpoints in total. Since every leaf must be an endpoint, $2k \geq |L|$, which gives $k \geq |L|/2$. $\square$

2.4 The Combined Bound

Putting all three bounds together, we get the final lower bound:

$$\text{mpc}(G) \geq \max \left(c, \sum_{j=1}^c d_j, \left\lceil \frac{|L|}{2} \right\rceil \right).$$

This is the theoretical floor. In the chessboard setting, the imbalance bound is usually the tightest one.

3 Problem Setup — The Chessboard Grid

Now we can proceed to find the algorithm to solve the problem(perhaps).

Take an $R \times C$ chessboard. Each cell (i, j) gets value $+1$ if $i + j$ is even (black) and -1 if $i + j$ is odd (white). The user may remove some cells, leaving a set of active cells V . Two cells are adjacent if they share a side (4-connectivity).

The goal is twofold:

1. Partition each connected component of V into connected subsets (subdomains) so that each subdomain has total black-white sum $S \in \{-1, 0, +1\}$. That is the white and black imbalance is atmost 1, a necessary condition to have a hamiltonian path.
2. Find a Hamiltonian path inside each subdomain, visiting every cell exactly once. If no full Hamiltonian path exists, return the longest partial path found.

4 Partition into Balanced Subdomains

4.1 Construct a Spanning Tree

For each connected component C containing n cells, we build a spanning tree using BFS/DFS on the adjacency graph. The tree is rooted at an arbitrary node (root r). Let $\text{children}(u)$ be the set of children of u in the tree.

4.2 Subtree Sums

Define $f(u)$ as the sum of black/white values (+1 for black, -1 for white) in the subtree rooted at u . Compute $f(u)$ via postorder traversal:

$$f(u) = \text{value}(u) + \sum_{v \in \text{children}(u)} f(v).$$

The total sum of the whole component is $f(r) = \delta_C$.

4.3 Cutting Subtrees to Obtain $|\delta_C|$ Subdomains

Assume $\delta_C > 0$ (the case $\delta_C < 0$ is symmetric by flipping signs). We need δ_C subdomains each with sum +1. We repeatedly cut an edge (p, c) where c is a child with $f(c) = +1$ and c is not yet assigned to any domain. Cutting removes the whole subtree rooted at c as a new domain. After cutting, we subtract the sum from all ancestors:

$$f(a) \leftarrow f(a) - (+1) \quad \text{for each ancestor } a \text{ of } c.$$

We continue until $\delta_C - 1$ such subtrees have been cut. The remaining nodes (including the root) form the final domain.

Correctness: This algorithm guarantees each domain is connected (subtree or what remains) and has sum exactly +1 (or -1 for negative components). For components with $\delta_C = 0$ we keep the whole component as one domain (sum 0).

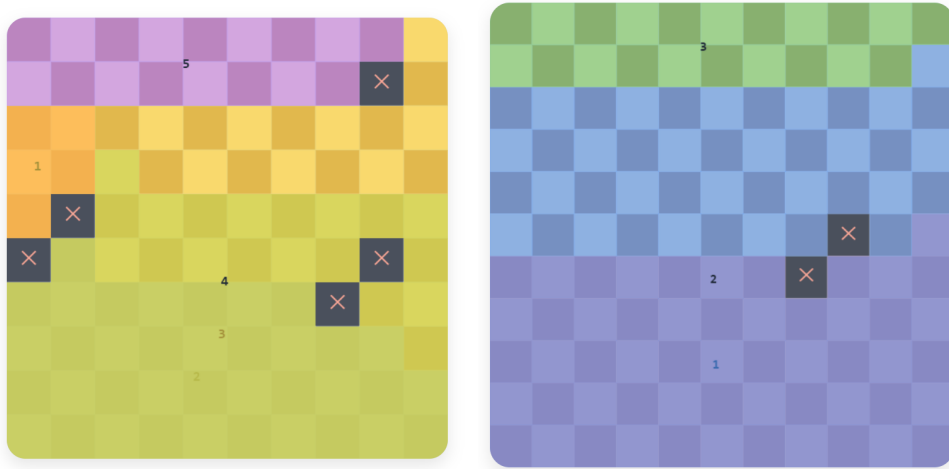


Figure 1: There are 5 imbalances of W-B in left and 3 imbalance for right. Hence domain is divided into corresponding number of connected parts

5 Finding Hamiltonian Paths

5.1 Stage 1 — Greedy Warnsdorf Heuristic

At each step, move to the unvisited neighbour that has the fewest unvisited neighbours itself. The intuition is: visit “dead-end” areas early, before they become unreachable.

5.2 Stage 2 — Depth-First Search with Backtracking

If Stage 1 fails to find a complete path, we switch to an iterative DFS. The neighbours at each step are still sorted in Warnsdorff order (fewest neighbours first), which guides the search toward good solutions earlier.

To keep things practical, we impose a time budget:

- For subdomains with $m \leq 46$ cells: up to 1.8 seconds.
- For larger subdomains: up to 4.5 seconds.

During the search we keep track of the longest path found so far.

5.3 Stage 3 — Partial Path Fallback

If no Hamiltonian path is found within the time budget, we return the longest partial path discovered. This ensures the tool always produces a usable result, even for subdomains that are theoretically hard.

5.4 Pruning for Speed

Before extending the current path by one step, we do a quick connectivity check on the remaining unvisited cells. If those cells have become disconnected, this branch can never lead to a full Hamiltonian path, so we prune it immediately. This cuts down the search space significantly.

6 Complexity and Practical Performance

Step	Complexity	Notes
Partitioning	$O(n)$ per component	BFS/DFS + one postorder pass
Hamiltonian search	Worst-case exponential	Tamed by heuristics and time limits

In practice, the algorithm handles grids up to 12×12 (144 cells) comfortably. For larger or sparser boards, the time limits and partial-path fallback ensure the tool never hangs.

7 Results and discussions

7.1 Planar case

There is nothing to discuss, except for a few figures.

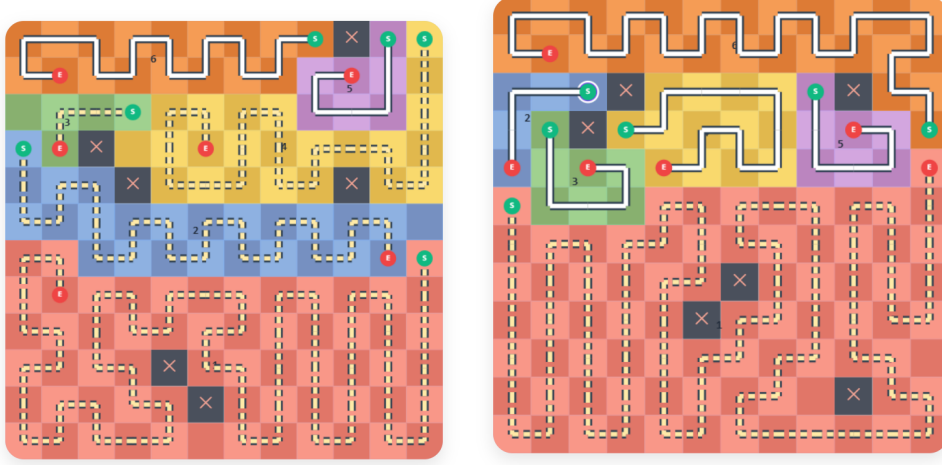


Figure 2: All the subdomain is almost have a Hamiltonian path and the entire domain has been covered with minimum number of paths

However, I think we need some discussion here.

First, although there is an algorithm to find subdomains with the necessary conditions, they are not optimized properly. If we could optimize them, we could be more certain that we can always cover the whole domain with the minimum number of paths. In our case, there are still 1–2 cells that are not traversed by the algorithm.

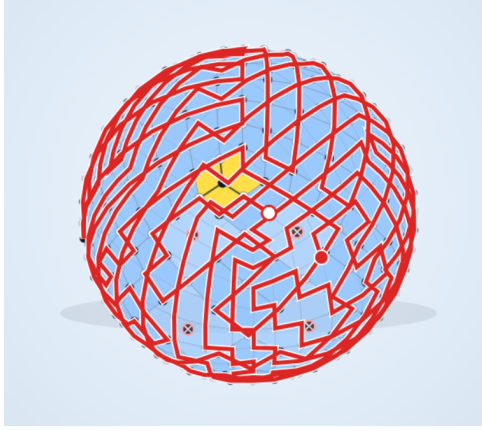
Second, if we restrict the minimum and maximum size a subdomain can take, it will be helpful for solving large domains, since we can then use a parallel solver for each subdomain. Also, the possibility of an ill-formed subdomain reduces with size.

Third, when holes are spaced closely, the normal Warnsdorff heuristic is effective, but when they are spaced irregularly, dividing the subdomain is efficient. (This is my thought only, no back up from mathematics).

The more about this is discussed in the future work section.

7.2 "Closed" surface case

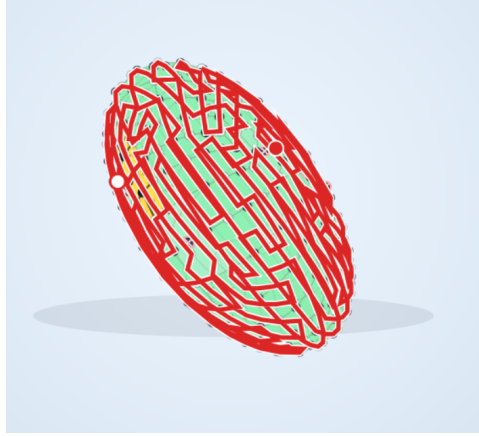
The same procedure and algorithm as mentioned above is applied to some standard closed manifolds (sphere, torus and ellipsoid) and results are shown below



(a) Sphere domain



(b) Torus domain



(c) ellipsoid domain

Figure 3: In three different (Topologically two) domains Hamiltonian paths are shown .

It was observed that these closed surfaces almost always exhibit a full Hamiltonian path. I am not sure about the explanation, but this may be because the boundary constraints are now removed. For example, in a torus, when we represent it as a rectangle with opposite sides identified, a path can 'teleport' from one side to the other without getting stuck at any location.

8 Future Work and Some discussions

In this section I tried to explain the future planing about the problem and some challenges that we might face and also possiblities of avoiding them.

Improving the Domain Division Algorithm

The current algorithm for partitioning a domain into balanced subdomains is not fully optimized. Future work should focus on developing a more efficient and robust division

strategy that guarantees (or comes closer to) covering the entire domain with the minimum number of paths. In particular, the algorithm should aim to eliminate cases where 1–2 cells remain uncovered.

Mesh Triangulation and Its Limitations

Using a triangular mesh is generally unsuitable for Hamiltonian path problems. Even a simple, simply connected domain, when subdivided into triangles, may lack a Hamiltonian path altogether.

Topology Independence and Arbitrary Holes

The current algorithm makes no use of geometric distances; it relies solely on graph connectivity and bipartite imbalance. Consequently, it applies to any topology. Holes can be placed arbitrarily, and the domain can be divided into a mesh (quadrilateral or mixed) and solved without modification.

Quadrilateral-to-Triangle Conversion

Triangulation can still be used if it is performed by first discretizing the domain with quadrilaterals and then splitting each quadrilateral along a diagonal to produce triangles. Conversely, a triangular mesh can be merged into quadrilaterals by pairing adjacent triangles. Such conversions allow the algorithm to handle mixed meshes while retaining the possibility of Hamiltonian paths.

Parallel Solver for Larger Domains

To scale up to very large domains, a parallel solver should be implemented. Different subdomains (produced by the partitioning step) can be solved independently on separate threads or computing nodes. This would significantly reduce runtime and make the approach feasible for high-resolution grids and complex geometries.

9 Summary

1. Compute the lower bound. For each connected component, find its imbalance d_j . Count the leaves $|L|$. The minimum number of paths needed is at least $\max(c, \sum d_j, \lceil |L|/2 \rceil)$.
2. Partition into balanced subdomains. Build a spanning tree per component. Cut subtrees of sum ± 1 greedily until each piece has imbalance at most 1.
3. Find Hamiltonian paths. For each subdomain, run Warnsdorff greedy, then DFS with backtracking and time limits, then fall back to the longest partial path.

The algorithm does not always find the globally optimal partition or path, but the lower bounds tell us how far we are from optimal. And the heuristics work well enough that for most realistic board shapes, full Hamiltonian paths are found in every subdomain.